

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

ОТЧЁТ

по учебной практике

УП.02.01 Технология разработки программного обеспечения

Профессионального модуля ПМ.02 Осуществление интеграции программных
модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ Ощепков Михаил Алексеевич
(подпись) (фамилия, имя, отчество)

Группы ИПо-32

Руководитель практической подготовки от профильной организации

Преподаватель _____
(должность)

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

МП «26» июня 2026 года

Руководитель практической подготовки от института

_____ Берестов Дмитрий Борисович
(подпись) (фамилия, имя, отчество)

«27» июня 2026 года

Пермь, 2026 год

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
1. ОПИСАНИЕ ЗАДАНИЯ	4
2. АРХИТЕКТУРА ПРИЛОЖЕНИЯ	5
2.1. Структура решения	5
2.2. Диаграмма классов	6
3. ПРОЦЕСС СОЗДАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	8
3.1. Создание структуры проекта	8
3.2. Разработка библиотеки бизнес-логики	8
3.3. Разработка пользовательского интерфейса	9
3.4. Написание модульных тестов	9
4. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ	11
4.1. Главное окно приложения	11
4.2. Добавление задачи	11
4.3. Редактирование задачи	12
4.4. Фильтрация по статусу	13
4.5. Поиск задач	13
4.6. Сортировка задач	13
4.7. Сохранение и загрузка файла	14
4.8. Просроченные задачи	14
ЗАКЛЮЧЕНИЕ	15

ВВЕДЕНИЕ

Настоящая сопроводительная записка составлена по результатам учебной практики, проходившей в рамках освоения дисциплины «Разработка программного обеспечения».

Целью практики являлось закрепление теоретических знаний и получение практических навыков разработки объектно-ориентированных приложений на языке C# с применением паттернов проектирования и современных инструментов разработки.

В ходе практики было разработано приложение OshchepkovTaskManager — настольное WPF-приложение для управления задачами, реализующее полный набор операций CRUD, фильтрацию, поиск, сортировку, сохранение данных в формате JSON и визуальную статистику.

Задание предусматривало разделение логики приложения и пользовательского интерфейса: основная бизнес-логика вынесена в отдельную динамическую библиотеку (DLL), а пользовательский интерфейс реализован в виде WPF-приложения. Для контроля качества кода написаны модульные тесты с использованием фреймворка xUnit.

Записка содержит диаграмму классов разработанного программного обеспечения, а также подробное описание процесса создания приложения с иллюстрациями.

1. ОПИСАНИЕ ЗАДАНИЯ

В рамках учебной практики требовалось разработать приложение для управления задачами. Задание включало следующие обязательные требования:

- добавление новой задачи с указанием названия, описания, приоритета, срока выполнения и статуса;
- просмотр списка всех задач;
- фильтрация задач по статусу («Новая», «В процессе», «Завершена», «Отменена»);
- поиск задач по названию или описанию;
- редактирование существующих задач;
- удаление задач;
- сохранение и загрузка задач в/из файла формата JSON;
- реализация основной логики в виде динамической библиотеки, пользовательского интерфейса — в виде WPF-приложения;
- написание модульных тестов для классов библиотеки.

Дополнительно были реализованы:

- сортировка задач по приоритету и сроку выполнения;
- пометка задачи как «Важная» (отображается звёздочкой и цветом);
- статистика по задачам (всего, новых, в процессе, завершено, просрочено).

Используемые технологии: язык программирования C#, платформа .NET 8, WPF (Windows Presentation Foundation), паттерн проектирования MVVM, фреймворк xUnit для тестирования, формат JSON для хранения данных. Среда разработки — Microsoft Visual Studio 2022.

2. АРХИТЕКТУРА ПРИЛОЖЕНИЯ

2.1. Структура решения

Решение OshchepkovTaskManager состоит из трёх проектов, каждый из которых выполняет строго определённую роль:

- OshchepkovTaskManager.Core — библиотека классов (DLL), содержащая всю бизнес-логику приложения: модели данных, интерфейсы и сервисы;
- OshchepkovTaskManager.App — WPF-приложение, реализующее пользовательский интерфейс на основе паттерна MVVM;
- OshchepkovTaskManager.Tests — проект с модульными тестами на базе фреймворка xUnit.

Такое разделение обеспечивает независимость бизнес-логики от конкретной реализации интерфейса, упрощает тестирование и сопровождение кода.

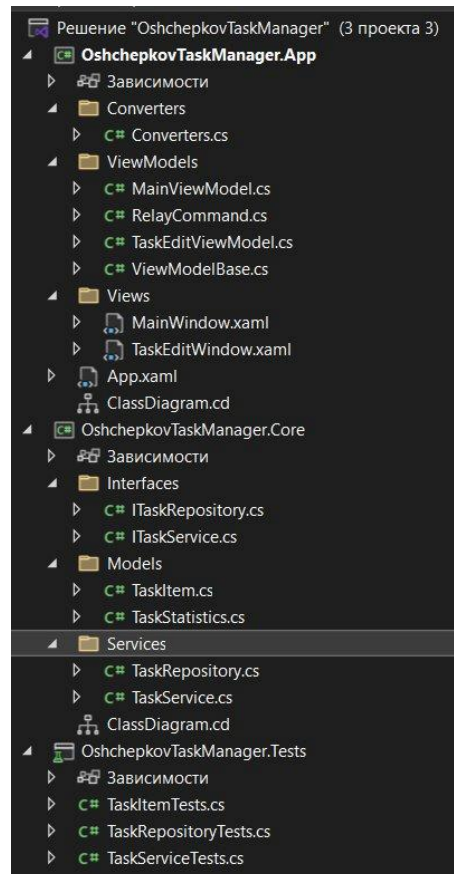


Рисунок 1 — Структура решения в Solution Explorer

2.2. Диаграмма классов

На рисунке 2 представлена диаграмма классов проекта OshcherkovTaskManager.Core, построенная средствами Visual Studio 2022. Диаграмма отражает состав и взаимосвязи основных классов и интерфейсов библиотеки.

Библиотека содержит следующие элементы:

- TaskItem — основная модель данных, описывающая задачу. Содержит свойства: Id, Title, Description, Priority, DueDate, Status, IsImportant, CreatedAt, IsOverdue;
- TaskStatistics — модель для хранения статистических данных (Total, New, InProgress, Completed, Cancelled, Overdue, Important);
- TaskPriority — перечисление приоритетов: Low, Medium, High, Critical;
- TaskStatus — перечисление статусов: New, InProgress, Completed, Cancelled;
- ITaskRepository — интерфейс репозитория, объявляющий методы работы с коллекцией задач и файловым вводом-выводом;
- ITaskService — интерфейс сервиса, объявляющий бизнес-методы (CRUD, поиск, фильтрация, сортировка, статистика);
- TaskRepository — конкретная реализация ITaskRepository с хранением данных в памяти и сериализацией в JSON;
- TaskService — конкретная реализация ITaskService, использующая LINQ для фильтрации, поиска и сортировки задач.

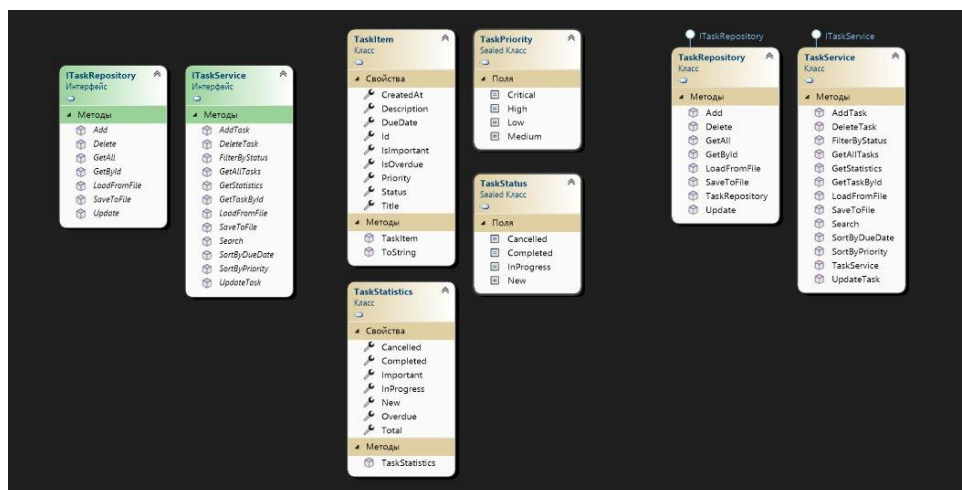


Рисунок 2 — Диаграмма классов OshcherkovTaskManager.Core

3. ПРОЦЕСС СОЗДАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

3.1. Создание структуры проекта

Разработка началась с создания решения (Solution) в Visual Studio 2022. В решение были добавлены три проекта: библиотека классов `OshchepkovTaskManager.Core` (.NET 8), WPF-приложение `OshchepkovTaskManager.App` (.NET 8, `net8.0-windows`) и проект тестов `OshchepkovTaskManager.Tests` (xUnit).

В библиотеке `Core` были созданы три папки: `Models` (модели данных), `Interfaces` (интерфейсы) и `Services` (реализации). В проекте `App` — папки `Views`, `ViewModels` и `Converters` согласно паттерну MVVM.

3.2. Разработка библиотеки бизнес-логики

Разработка библиотеки началась с определения модели данных. Класс `TaskItem` описывает одну задачу и содержит все необходимые свойства: уникальный идентификатор (`Guid`), название, описание, приоритет, срок выполнения, статус, флаг важности и дату создания. Вычисляемое свойство `IsOverdue` автоматически определяет, является ли задача просроченной.

Для хранения статистики создан класс `TaskStatistics` с полями-счётчиками для каждого статуса и дополнительными полями `Overdue` и `Important`.

Интерфейсы `ITaskRepository` и `ITaskService` определяют контракты для работы с данными и бизнес-логикой соответственно. Такой подход обеспечивает слабую связанность компонентов и возможность замены реализации без изменения зависимого кода.

Класс `TaskRepository` реализует `ITaskRepository`: хранит задачи в коллекции `List<TaskItem>` в оперативной памяти, а для сохранения и загрузки использует `System.Text.Json` с настройкой `JsonStringEnumConverter` для читаемого формата файла.

Класс `TaskService` реализует `ITaskService` и содержит всю бизнес-логику: методы поиска и фильтрации используют LINQ-запросы, метод `GetStatistics` подсчитывает статистику одним проходом по коллекции.

3.3. Разработка пользовательского интерфейса

Пользовательский интерфейс разработан по паттерну MVVM (Model-View-ViewModel). Базовый класс `ViewModelBase` реализует интерфейс `INotifyPropertyChanged`, обеспечивая автоматическое обновление интерфейса при изменении данных. Класс `RelayCommand` реализует `ICommand` для привязки команд в XAML.

Главное окно приложения (`MainWindow`) имеет макет с боковой панелью и основной рабочей областью. Боковая панель содержит кнопки управления задачами и блок статистики. Основная область включает строку поиска и фильтрации, заголовки таблицы и прокручиваемый список задач.

Для отображения строк задач используется `ItemsControl` с кастомным `DataTemplate`, что обеспечивает гибкое управление внешним видом каждой строки. Просроченные задачи выделяются тёмно-красным фоном через триггер `DataTrigger`.

Конвертеры значений (`IValueConverter`) в папке `Converters` преобразуют перечисления в локализованные строки и цвета для отображения бейджей приоритета и статуса.

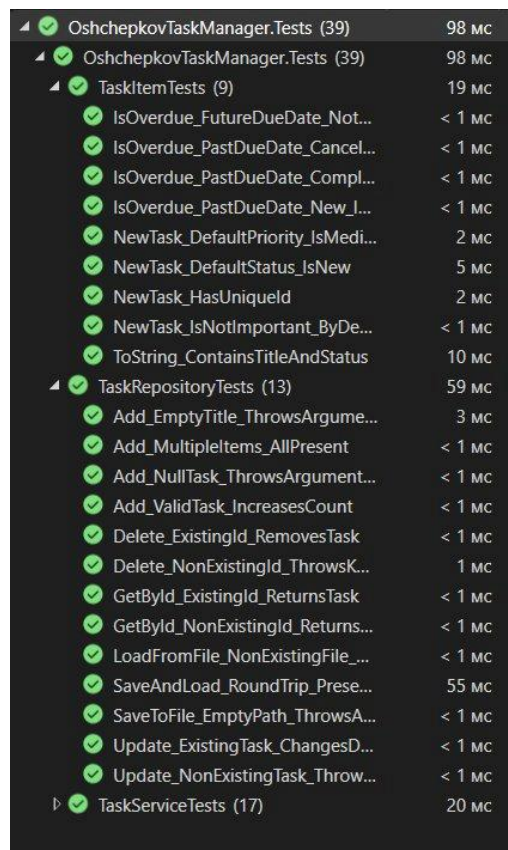
Диалоговое окно `TaskEditWindow` используется как для добавления, так и для редактирования задач. Его `ViewModel` — `TaskEditViewModel` — принимает существующую задачу или `null` и инициализирует поля формы соответствующим образом.

3.4. Написание модульных тестов

Для обеспечения качества кода были написаны модульные тесты с использованием фреймворка `xUnit`. Тесты охватывают три класса: `TaskItem`, `TaskRepository` и `TaskService`.

Тесты проверяют корректность основных операций: создание задачи с уникальным идентификатором, добавление и получение задач, обновление и удаление, корректную обработку исключительных ситуаций (null-аргументы, несуществующие записи, отсутствующий файл), сохранение и загрузку данных из файла, поиск и фильтрацию с учётом регистра, сортировку, подсчёт статистики и вычисление свойства IsOverdue.

Всего написано 39 тестов, все тесты проходят успешно.



▲ ✓ OshchepkovTaskManager.Tests (39)	98 мс
▲ ✓ OshchepkovTaskManager.Tests (39)	98 мс
▲ ✓ TaskItemTests (9)	19 мс
✓ IsOverdue_FutureDueDate_Not...	< 1 мс
✓ IsOverdue_PastDueDate_Cancel...	< 1 мс
✓ IsOverdue_PastDueDate_Compl...	< 1 мс
✓ IsOverdue_PastDueDate_New_I...	< 1 мс
✓ NewTask_DefaultPriority_IsMedi...	2 мс
✓ NewTask_DefaultStatus_IsNew	5 мс
✓ NewTask_HasUniqueld	2 мс
✓ NewTask_IsNotImportant_ByDe...	< 1 мс
✓ ToString_ContainsTitleAndStatus	10 мс
▲ ✓ TaskRepositoryTests (13)	59 мс
✓ Add_EmptyTitle_ThrowsArgume...	3 мс
✓ Add_MultipleItems_AllPresent	< 1 мс
✓ Add_NullTask_ThrowsArgument...	< 1 мс
✓ Add_ValidTask_IncreasesCount	< 1 мс
✓ Delete_ExistingId_RemovesTask	< 1 мс
✓ Delete_NonExistingId_ThrowsK...	1 мс
✓ GetById_ExistingId_ReturnsTask	< 1 мс
✓ GetById_NonExistingId_Returns...	< 1 мс
✓ LoadFromFile_NonExistingFile_...	< 1 мс
✓ SaveAndLoad_RoundTrip_Prese...	55 мс
✓ SaveToFile_EmptyPath_ThrowsA...	< 1 мс
✓ Update_ExistingTask_ChangesD...	< 1 мс
✓ Update_NonExistingTask_Throw...	< 1 мс
▶ ✓ TaskServiceTests (17)	20 мс

Рисунок 3 — Результаты запуска модульных тестов (39 тестов пройдено)

4. РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

4.1. Главное окно приложения

При запуске приложения открывается главное окно, разделённое на две области: боковую панель управления (слева) и рабочую область со списком задач (справа). В боковой панели расположены кнопки основных операций и блок статистики. По умолчанию список содержит шесть демонстрационных задач.

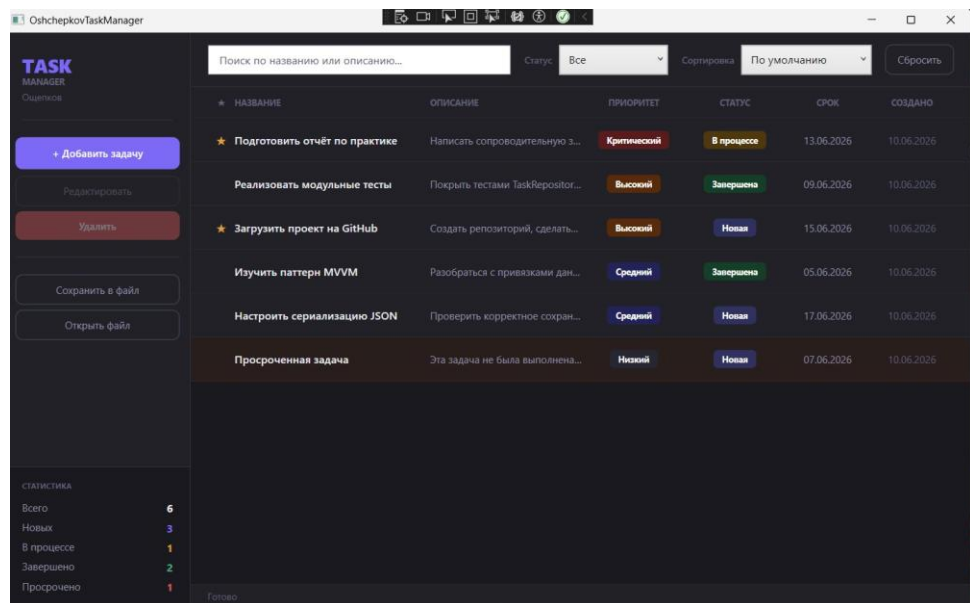


Рисунок 4 — Главное окно приложения OshchepkovTaskManager

4.2. Добавление задачи

Для добавления новой задачи необходимо нажать кнопку «+ Добавить задачу» в боковой панели. Откроется диалоговое окно «Новая задача», в котором нужно заполнить поля: название (обязательное), описание, приоритет, статус и срок выполнения. При необходимости можно включить переключатель «Важная задача». После заполнения полей нажать кнопку «Сохранить».

Задача

Новая задача

Заполните поля и нажмите Сохранить

НАЗВАНИЕ *

Реализовать модульные тесты

ОПИСАНИЕ

Покрыть тестами TaskRepository и TaskService

ПРИОРИТЕТ

Средний

СТАТУС

В процессе

СРОК ВЫПОЛНЕНИЯ

Отмена Сохранить

Рисунок 5 — Диалоговое окно добавления новой задачи

4.3. Редактирование задачи

Для редактирования задачи необходимо выбрать её в списке одиночным щелчком мыши, после чего нажать кнопку «Редактировать» в боковой панели (либо выполнить двойной щелчок по строке задачи). Откроется диалоговое окно «Редактировать задачу» с предзаполненными текущими значениями. После внесения изменений нажать «Сохранить».

Задача

Редактировать задачу

Внесите изменения и нажмите Сохранить

НАЗВАНИЕ *

Подготовить отчёт по практике

ОПИСАНИЕ

Написать сопроводительную записку, диаграмму классов и описание ПО

ПРИОРИТЕТ

Критический

СТАТУС

В процессе

СРОК ВЫПОЛНЕНИЯ

Отмена Сохранить

Рисунок 6 — Диалоговое окно редактирования задачи

4.4. Фильтрация по статусу

Для отображения задач с определённым статусом необходимо выбрать нужное значение в выпадающем списке «Статус» на панели фильтрации. Доступные значения: «Все», «Новая», «В процессе», «Завершена», «Отменена». Список задач автоматически обновится.

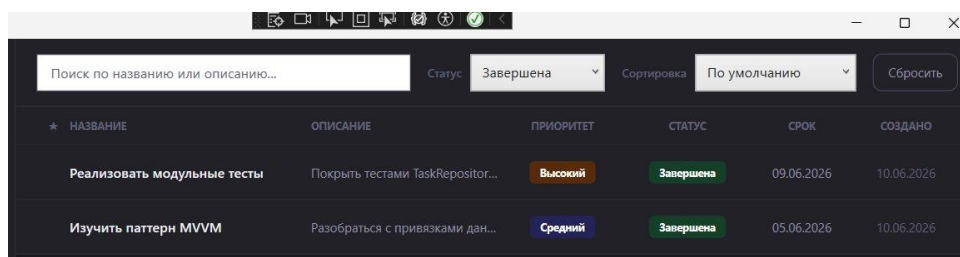


Рисунок 7 — Фильтрация задач по статусу «Завершена»

4.5. Поиск задач

Для поиска задач необходимо ввести текст в поле «Поиск по названию или описанию...». Поиск выполняется в реальном времени по мере ввода и охватывает как название, так и описание задачи. Поиск регистронезависимый.

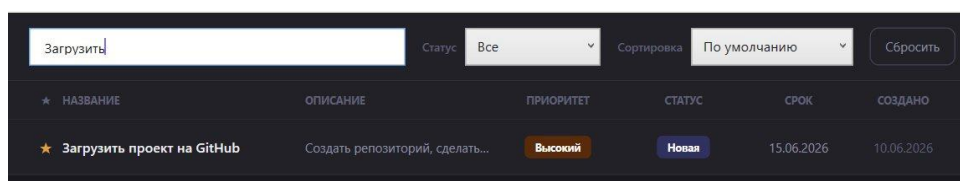


Рисунок 8 — Результат поиска по слову «Загрузить»

4.6. Сортировка задач

Для сортировки списка задач необходимо выбрать нужное значение в выпадающем списке «Сортировка». Доступные варианты: «По умолчанию», «По приоритету ↓», «По приоритету ↑», «По сроку ↑», «По сроку ↓».

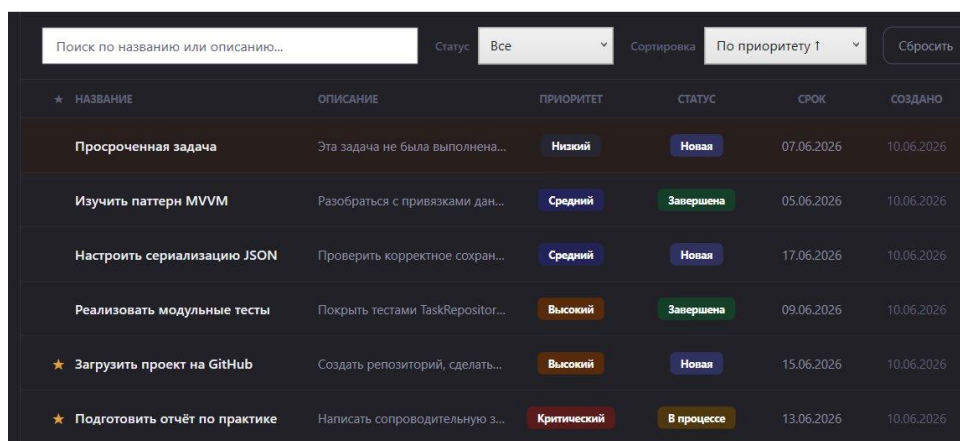


Рисунок 9 — Список задач, отсортированных по приоритету по возрастанию

4.7. Сохранение и загрузка файла

Для сохранения задач в файл необходимо нажать кнопку «Сохранить в файл» в боковой панели. Откроется стандартный диалог сохранения файла. Данные сохраняются в формате JSON. Для загрузки ранее сохранённых задач нажать «Открыть файл» и выбрать JSON-файл.



Рисунок 10 — Диалог сохранения задач в JSON-файл

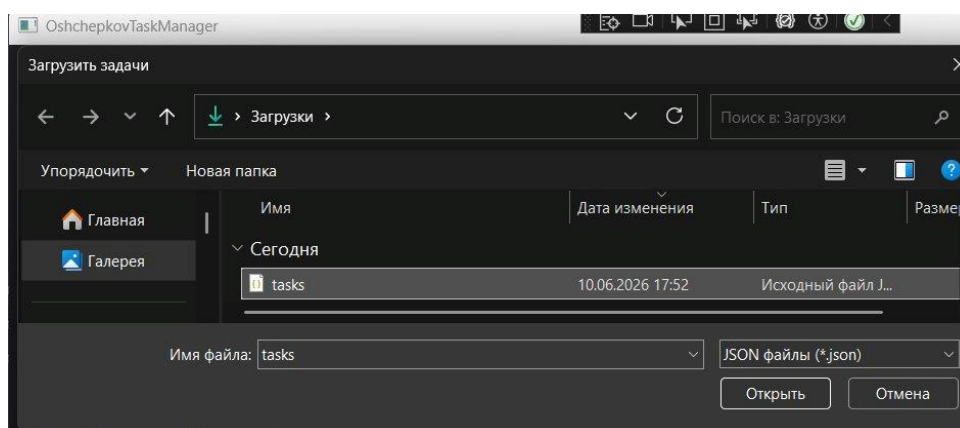


Рисунок 11 — Диалог загрузки задач из JSON-файла

4.8. Просроченные задачи

Задачи, срок выполнения которых истёк, а статус не является «Завершена» или «Отменена», автоматически выделяются тёмно-красным фоном в списке. Счётчик просроченных задач отображается в блоке статистики боковой панели.

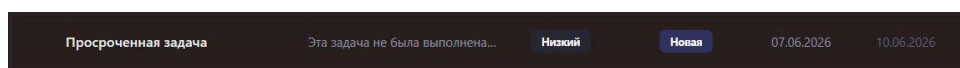


Рисунок 12 — Отображение просроченной задачи в списке

ЗАКЛЮЧЕНИЕ

В ходе учебной практики было разработано приложение `OshchepkovTaskManager` — полнофункциональная система управления задачами, реализованная на языке `C#` с использованием платформы `.NET 8` и технологии `WPF`.

В процессе разработки были выполнены все обязательные требования задания: реализованы операции создания, просмотра, редактирования и удаления задач, фильтрация по статусу, поиск по тексту, сохранение и загрузка данных в формате `JSON`. Дополнительно реализованы сортировка по приоритету и сроку, пометка задач как важных и подсчёт статистики.

Архитектурное решение с разделением на библиотеку бизнес-логики (`OshchepkovTaskManager.Core`) и `WPF`-приложение (`OshchepkovTaskManager.App`) обеспечивает высокую степень независимости компонентов. Применение паттерна `MVVM` в пользовательском интерфейсе гарантирует чёткое разделение представления и логики.

Написанные модульные тесты (39 тестов) подтверждают корректность реализации всех ключевых функций библиотеки. Все тесты успешно прошли проверку.

В результате практики были закреплены навыки объектно-ориентированного программирования, работы с коллекциями и `LINQ`, применения интерфейсов и паттернов проектирования, разработки `WPF`-приложений по паттерну `MVVM`, а также написания модульных тестов.